NAAN MUDHALVAN SMART BRIDGE

FITBUTTY-AI FITNESS PLAN GENERTOR USING GEMINI MODELS

STUDENTNAME : Masilamani.V, Shalini.R,Abinaya.R

YEAR : III

COLLEGE CODE : BRU5X

COLLEGE NAME : ST.PAUL'S COLLEGE OF ARTS AND SCIENCE FOR WOMEN

DEPARTMENT : COMPUTER SCIENCE

SEMESTER :III

ROLL NO : 2522K1191, 2522K1193, 2522K1184

NMID : 3E16F53745553679409489EFF21AF991, C5C950ADD3AD0E6B26E41929D732C95E, 401DBEF8005ACEABDD0430071773182B

DATE :28/09/2026

TITLE : FITBUDDY -AI FITNESS PLAN GENERATOR USING GEMINI MODELS

GITHUB LINK :abinayaravi131-afk https://share.google/s3qG7mu8xSQiS7Zi7

1. Project Overview

FitBuddy is a user-focused web application that generates a personalized 7-day workout plan from user information such as name, age, weight, fitness goal, and workout intensity. It also provides a nutrition or

recovery tip and supports feedback-based plan updates. A separate view allows administrators or developers to inspect stored user plans.

2. Problem Statement

Many users need structured fitness guidance but may not know how to create a weekly routine that matches their goals and preferred intensity. FitBuddy addresses this by combining a simple web form, generative AI, and persistent storage to produce a structured plan and revise it from user feedback.

3. Objectives

• Generate personalized 7-day workout plans.
• Provide concise nutrition/recovery guidance.
• Accept feedback and revise an existing plan.
• Store user information and plan history.
• Provide a clear browser-based interface.
• Demonstrate FastAPI, Jinja2, Gemini integration, SQLite, and SQLAlchemy in one project.

4. Project Scenarios

Scenario 1: A user enters personal and fitness details and requests a plan.
Scenario 2: A user provides feedback such as requesting more cardio or additional recovery, and the application generates a revised plan.
Scenario 3: The application generates a concise nutrition or recovery tip based on the selected goal.
Scenario 4: An administrator/developer opens the all-users page to inspect user details and original/updated plans.

5. Technical Architecture

The application follows a modular architecture: HTML/Jinja2 templates form the presentation layer; FastAPI handles routes and request processing; Gemini modules provide AI-generated content; SQLite with SQLAlchemy stores user and plan information.

6. Project Workflow / Milestones

Milestone 1 – Model selection and architecture
• Understand AI requirements

• Define frontend, backend, AI and database layers
• Prepare the development environment

Milestone 2 – Core functionality
• Workout generation
• Nutrition/recovery tip generation
• Feedback-based plan updating
• User and plan persistence

Milestone 3 – routes.py development
• Home route
• Workout generation route
• Feedback route
• Admin/all-users route

Milestone 4 – Frontend development
• index.html
• result.html
• all_users.html
• Responsive and readable form/result presentation

Milestone 5 – Deployment and verification
• Install dependencies
• Configure environment variables
• Run with Uvicorn
• Verify browser pages and FastAPI /docs

7. Environment Setup

Create and activate a virtual environment, install requirements, configure GOOGLE_API_KEY, and run the FastAPI application with Uvicorn. The project also contains a demo fallback so the interface can be demonstrated without an API key; live Gemini generation requires a valid configured API key.

8. Project Structure

FitBuddy_Final_Project/
├── app/

```
│   ├── __init__.py
│   ├── main.py
│   ├── routes.py
│   ├── database.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   └── updated_plan.py
├── templates/
│   ├── index.html
│   ├── result.html
│   └── all_users.html
├── static/
│   ├── output_home.png
│   └── output_result.png
├── requirements.txt
├── .env.example
├── .gitignore
├── README.md
└── sample_output.txt
```

9. Core Modules

app/main.py – creates the FastAPI application and registers routes. app/routes.py – connects browser forms to AI and database operations. app/database.py – defines the SQLAlchemy model and persistence helpers.

app/gemini_generator.py – generates the 7-day workout plan.

app/gemini_flash_generator.py – generates the nutrition/recovery tip.

app/updated_plan.py – revises the plan from user feedback.

templates/index.html – input page. templates/result.html – workout, tip, and feedback page.

templates/all_users.html – stored-user/admin view.

10. Functional Flow

1. User opens the home page.

2. User enters name, ID, age, weight, goal, and intensity.
3. FastAPI receives the form.
4. Workout generation and nutrition-tip modules are called.
5. User information and plan are stored in SQLite.
6. result.html displays the personalized output.
7. User may submit feedback.
8. The original plan plus feedback is passed to the update module.
9. The revised plan is stored and displayed.
10. The all-users page displays stored records.

11. Frontend Pages

Home page: collects username, user ID, age, weight, goal and intensity.

Result page: shows user details, workout plan, nutrition/recovery tip, and feedback form.

All-users page: displays saved records and original/updated plan information.

12. Database Design

The User model stores user_id, username, age, weight, goal, intensity, original_plan, updated_plan, and nutrition_tip. The database layer provides functions for saving users/plans, retrieving a user, updating a plan, and retrieving all users.

13. Testing Plan

Test valid form submission, missing fields, repeated user IDs, feedback submission, unknown user IDs, database persistence, demo fallback behavior, and page rendering. For live AI testing, verify that GOOGLE_API_KEY is configured and that the configured Gemini service is reachable.

14. Output / Demonstration

The package includes demonstration interface screenshots in static/output_home.png and static/output_result.png and a sample_output.txt file. These are presentation/demo artifacts; they should not be described as live Gemini responses unless the application is actually run with a valid API key.

15. Deployment

Windows setup:

python -m venv venv

venv\Scripts\activate pip

install -r requirements.txt

uvicorn app.main:app --

reload

Then open http://127.0.0.1:8000 and the FastAPI documentation at http://127.0.0.1:8000/docs.

16.Installation in VS Code

Open the project folder in VS Code and run:

python -m

venv .venv .\.venv\Scripts\Activate.ps1 pip

install -r requirements.txt

copy .env.example .env uvicorn

app.main:app --reload

Then open http://127.0.0.1:8000

17. Security and Responsible Use

Keep API credentials in environment variables and never commit real keys. Fitness suggestions are general informational content and should not be treated as medical diagnosis or individualized medical advice. Users should adjust activity to their abilities and seek professional advice where appropriate.

18. Limitations

The project is a demonstration application. AI output can vary, the demo fallback is not a live AI response, and there is no full authentication/authorization layer in this final local package. The application is intended as a project demonstration rather than a clinical fitness platform.

19. Future Enhancements

Authentication and role-based access, richer exercise libraries, progress charts, plan export to PDF, stronger validation, cloud database support, production logging, automated tests, and a current Gemini SDK/API implementation can be added in future versions.

20. Conclusion

FitBuddy demonstrates an end-to-end AI web application in which a user submits fitness information, receives a structured plan and supporting tip, provides feedback, and can see an updated plan. The implementation combines FastAPI, Jinja2, Gemini integration, SQLite, and SQLAlchemy in a modular project structure.

21. Complete Code Files

Actual source files included in the final project package are reproduced below for documentation and submission.

1.app/main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI(
    title="FitBuddy – AI Fitness Plan Generator",
    description="AI-powered personalized fitness planning using Gemini models.",
version="1.0.0"
)
app.include_router(router)

@app.get("/health")
def health():
    return {"status": "ok", "application": "FitBuddy"}
```

2.app/routes.py

```
from fastapi import APIRouter, Request, Form from fastapi.responses
import HTMLResponse from fastapi.templating import
Jinja2Templates from .gemini_generator import
generate_workout_gemini from .gemini_flash_generator import
```

```
generate_nutrition_tip_with_flash from .updated_plan import
update_workout_plan
from .database import save_user, get_user, get_all_users, update_plan

router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.get("/", response_class=HTMLResponse) def
home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate-workout", response_class=HTMLResponse)
def generate_workout(     request: Request,     username: str =
Form(...),     user_id: str = Form(...),     age: int = Form(...),     weight:
str = Form(...),     goal: str = Form(...),     intensity: str = Form(...) ):
    existing = get_user(user_id)
    plan = generate_workout_gemini(username, age, weight, goal, intensity)
tip = generate_nutrition_tip_with_flash(goal)
    if existing:
        # Keep one user record and refresh the original plan.
        from .database import save_plan
save_plan(user_id, plan, tip)     else:
        save_user({
            "user_id": user_id, "username": username, "age": age,
            "weight": weight, "goal": goal, "intensity": intensity,
            "original_plan": plan, "updated_plan": None, "nutrition_tip": tip
        })
    return templates.TemplateResponse("result.html", {
        "request": request, "username": username, "user_id": user_id,
        "age": age, "weight": weight, "goal": goal, "intensity": intensity,
        "workout_plan": plan, "nutrition_tip": tip, "updated": False
    })
```

```
@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback(     request: Request,     user_id: str = Form(...),
feedback: str = Form(...)
):
   user = get_user(user_id)
if not user:
       return templates.TemplateResponse("result.html", {
          "request": request, "error": "User ID not found. Please generate a plan first."
       })
   revised = update_workout_plan(user.original_plan, feedback)
update_plan(user_id, revised)
   return templates.TemplateResponse("result.html", {
       "request": request, "username": user.username, "user_id": user.user_id,
       "age": user.age, "weight": user.weight, "goal": user.goal,
       "intensity": user.intensity, "workout_plan": revised,
       "nutrition_tip": user.nutrition_tip, "updated": True
   })

@router.get("/view-all-users", response_class=HTMLResponse) def
view_all_users(request: Request):
   return templates.TemplateResponse("all_users.html", {
       "request": request, "users": get_all_users()
   })
```

3.app/database.py

```
from sqlalchemy import create_engine, Column, Integer, String, Text from
sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "sqlite:///./fitbuddy.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False) Base
= declarative_base()
```

```
class User(Base):
__tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(50), unique=True, index=True, nullable=False)
username = Column(String(100), nullable=False)     age = Column(Integer,
nullable=False)     weight = Column(String(20), nullable=False)     goal =
Column(String(100), nullable=False)     intensity = Column(String(20),
nullable=False)     original_plan = Column(Text, nullable=False)
updated_plan = Column(Text, nullable=True)     nutrition_tip =
Column(Text, nullable=True)

Base.metadata.create_all(bind=engine)

def save_user(data):
db = SessionLocal()
try:
        user = User(**data)
db.add(user)
db.commit()
db.refresh(user)
return user     finally:
        db.close()

def save_plan(user_id, plan, nutrition_tip=None):
    db = SessionLocal()
try:
        user = db.query(User).filter(User.user_id == user_id).first()
if user:
            user.original_plan = plan
user.nutrition_tip = nutrition_tip
db.commit()     finally:
        db.close()
```

```
def update_plan(user_id, updated_plan):
    db = SessionLocal()
try:
        user = db.query(User).filter(User.user_id == user_id).first()
if user:
            user.updated_plan = updated_plan
db.commit()             return user     finally:
        db.close()

def get_user(user_id):
    db = SessionLocal()
try:
        return db.query(User).filter(User.user_id == user_id).first()
finally:
        db.close()

def get_all_users():
db = SessionLocal()
try:
        return db.query(User).order_by(User.id.desc()).all()
finally:
        db.close()
```

4.app/gemini_generator.py

```
import os

try:
    import google.generativeai as genai
except ImportError:     genai = None

def _demo_plan(goal, intensity):
    return f"""7-DAY PERSONALIZED WORKOUT PLAN
```

Goal: {goal.title()}
Intensity: {intensity.title()}

Day 1 – Full Body
Warm-up: 5–10 minutes brisk walking and mobility
Main workout: Squats 3x12, Push-ups 3x10, Glute bridges 3x15
Cooldown: 5 minutes stretching

Day 2 – Cardio
Warm-up: 5–10 minutes
Main workout: Brisk walk/jog 25 minutes + jumping jacks 3x20
Cooldown: 5 minutes easy walking

Day 3 – Upper Body
Warm-up: Shoulder and arm mobility
Main workout: Incline push-ups 3x10, rows 3x12, shoulder press 3x10
Cooldown: Upper-body stretching

Day 4 – Recovery
Light walking 20–30 minutes and full-body mobility

Day 5 – Lower Body
Warm-up: 5–10 minutes
Main workout: Squats 3x12, lunges 3x10 each leg, calf raises 3x15
Cooldown: Lower-body stretching

Day 6 – Core + Cardio
Warm-up: 5–10 minutes
Main workout: Plank 3x30 sec, dead bug 3x10, brisk walk 20 minutes Cooldown:
Breathing and stretching

Day 7 – Active Recovery
Easy walk, gentle stretching and recovery.

```
Safety note: Adjust intensity to your ability and stop if you experience pain or unusual symptoms."""

def generate_workout_gemini(name, age, weight, goal, intensity):
    api_key = os.getenv("GOOGLE_API_KEY")
if not api_key or genai is None:        return
_demo_plan(goal, intensity)
genai.configure(api_key=api_key)
    model = genai.GenerativeModel("gemini-1.5-pro")
prompt = f"""
Create a structured 7-day fitness plan for:
Name: {name}
Age: {age}
Weight: {weight} kg
Goal: {goal}
Workout intensity: {intensity}

Include for each day: focus, 5–10 minute warm-up, exercises with sets/reps or duration, and
cooldown/recovery guidance. Keep the response practical and clearly formatted.
Do not diagnose medical conditions.
"""
    response = model.generate_content(prompt)
return response.text
```

5.app/gemini_flash_generator.py

```
import os

try:
    import google.generativeai as genai except
ImportError:
    genai = None
```

```
def generate_nutrition_tip_with_flash(goal):
api_key = os.getenv("GOOGLE_API_KEY")     if
not api_key or genai is None:
       return f"Nutrition tip for {goal}: Prioritize balanced meals, adequate protein, vegetables, whole
grains, and regular hydration. Match portions to your energy needs."
genai.configure(api_key=api_key)
   model = genai.GenerativeModel("gemini-1.5-flash")
   prompt = f"Give one concise, practical nutrition or recovery tip for a person whose fitness goal is
{goal}. Avoid medical claims."
   return model.generate_content(prompt).text
```

6.app/updated_plan.py

```
import os

try:
   import google.generativeai as genai except
ImportError:
   genai = None

def update_workout_plan(original_plan, feedback):
api_key = os.getenv("GOOGLE_API_KEY")     if
not api_key or genai is None:
       return original_plan + f"\n\nUPDATED USING USER FEEDBACK:\n{feedback}\n\nThe revised
plan should incorporate the requested preference while keeping a balanced weekly schedule."
genai.configure(api_key=api_key)
   model = genai.GenerativeModel("gemini-1.5-pro")
prompt = f"""
Original workout plan:
{original_plan}

User feedback:
{feedback}
```

Revise the plan to incorporate the feedback while maintaining a structured 7-day schedule, appropriate progression, recovery, and clear warm-up/cooldown guidance.
"""
    return model.generate_content(prompt).text

7.templates/index.html

```
<!doctype html>
<html>
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
<title>FitBuddy – AI Fitness Plan Generator</title>
<style>
body{font-family:Arial,sans-
serif;margin:0;background:lineargradient(135deg,#111827,#1f2937);color:#fff;min-height:100vh}
.card{max-width:720px;margin:45px
auto;padding:32px;background:#ffffff;color:#111827;borderradius:18px;box-shadow:0 15px 40px #0008}
h1{margin-top:0;color:#0f766e}.grid{display:grid;grid-template-columns:1fr 1fr;gap:16px} label{font-
weight:700}input,select{width:100%;padding:11px;margin-top:6px;border:1px solid
#cbd5e1;border-radius:9px;box-sizing:border-box}
.full{grid-column:1/-1}button{background:#0f766e;color:white;border:0;padding:13px
22px;borderradius:10px;font-weight:700;cursor:pointer}
@media(max-width:650px){.grid{grid-template-columns:1fr}.full{grid-column:auto}}
</style>
</head>
<body><div class="card">
<h1>FitBuddy</h1><p>AI Fitness Plan Generator using Gemini Models</p>
<form method="post" action="/generate-workout"><div class="grid">
<div><label>Name</label><input name="username" required></div>
<div><label>User ID</label><input name="user_id" required></div>
<div><label>Age</label><input name="age" type="number" min="13" max="100" required></div>
<div><label>Weight (kg)</label><input name="weight" required></div>
```

<label>Fitness Goal</label><select name="goal"><option>Weight Loss</option><option>Muscle Gain</option><option>General Wellness</option><option>Flexibility</option></select>
<label>Workout Intensity</label><select name="intensity"><option>Low</option><option>Medium</option><option>High</option></select> <button type="submit">Generate Plan</button>
</form> </body></html>

8.templates/result.html

<!doctype html><html><head><meta charset="utf-8"><meta name="viewport" content="width=devicewidth,initial-scale=1">
<title>FitBuddy Result</title>
<style>
body{font-family:Arial;background:#f1f5f9;color:#0f172a;margin:0}.wrap{maxwidth:1000px;margin:30px auto;padding:20px}
.card{background:white;border-radius:16px;padding:22px;margin-bottom:20px;box-shadow:0 5px 20px #0001}
pre{white-space:pre-wrap;background:#0f172a;color:#e2e8f0;padding:18px;border-radius:12px;lineheight:1.5}
input,textarea{width:100%;padding:12px;box-sizing:border-box;border:1px solid #cbd5e1;borderradius:9px}
button{background:#0f766e;color:white;border:0;padding:12px 20px;border-radius:9px;font-weight:700}
.badge{display:inline-block;padding:6px 10px;border-radius:20px;background:#ccfbf1;color:#115e59}
</style></head><body>
<h1>Personalized Fitness Plan</h1>
Name: {{ username }}   User ID: {{ user_id }}
Age: {{ age }}   Weight: {{ weight }} kg
Goal: {{ goal }}   Intensity: {{ intensity }}
{% if updated %} Plan updated successfully using your feedback. {% endif %}
<h2>7-Day Workout Plan</h2><pre>{{ workout_plan }}</pre>
<h2>Nutrition / Recovery Tip</h2> {{ nutrition_tip }}
<h2>Improve Your Plan</h2>

```
<form method="post" action="/submit-feedback"><input type="hidden" name="user_id"
value="{{ user_id }}">
<label>Feedback</label><textarea name="feedback" rows="4" placeholder="Example: Include more
cardio or add more rest days." required></textarea>
<br><br><button type="submit">Update Plan</button></form></div>
<p><a href="/">Create another plan</a> | <a href="/view-all-users">View all users</a></p>
</div></body></html>
```

9.templates/all_users.html

```
<!doctype html><html><head><meta charset="utf-8"><title>FitBuddy Admin</title>
<style>body{font-
family:Arial;background:#f8fafc;margin:0}.wrap{padding:25px}table{width:100%;bordercollapse:collap
se;background:white}th,td{border:1px solid
#cbd5e1;padding:10px;verticalalign:top}th{background:#0f766e;color:white}pre{white-space:pre-
wrap;maxwidth:420px}</style></head>
<body><div class="wrap"><h1>FitBuddy – View All Users</h1>
<table><tr><th>User
ID</th><th>Name</th><th>Age</th><th>Weight</th><th>Goal</th><th>Intensity</th><th>Original
Plan</th><th>Updated Plan</th></tr>
{% for user in
users %}<tr><td>{{ user.user_id }}</td><td>{{ user.username }}</td><td>{{ user.age }}</td><td>{{ us
er.weight }}</td><td>{{ user.goal }}</td><td>{{ user.intensity }}</td><td><pre>{{ user.original_plan }}
</pre></td><td><pre>{{ user.updated_plan or "Not updated"
}}</pre></td></tr>{% endfor %}
</table></div></body></html>
```

10.requirements.txt

```
fastapi uvicorn
jinja2 sqlalchemy
python-multipart
google-generativeai
python-dotenv
```

11. .env.example

GOOGLE_API_KEY=your_gemini_api_key_here

12.README.md

FitBuddy – AI Fitness Plan Generator using Gemini Models

Development environment

Designed to be opened and run in Visual Studio / VS Code.

Features

- User registration/input form
- AI-generated 7-day workout plan
- Gemini Pro for workout generation and feedback-based updating
- Gemini Flash for concise nutrition/recovery tips
- SQLite + SQLAlchemy persistence
- Jinja2 frontend
- Admin view of users and original/updated plans
- FastAPI `/docs` interactive API documentation
- Demo fallback output when no Gemini API key is configured

Run ```bash

python -m venv

venv # Windows

venv\Scripts\activate #

macOS/Linux source

venv/bin/activate

pip install -r requirements.txt

copy .env.example .env

add GOOGLE_API_KEY to .env if using Gemini uvicorn

app.main:app --reload

```

Open `http://127.0.0.1:8000` and `http://127.0.0.1:8000/docs`.

## Project structure
app/main.py app/routes.py
app/database.py
app/gemini_generator.py
app/gemini_flash_generator.py
app/updated_plan.py
templates/index.html
templates/result.html
templates/all_users.html
requirements.txt .env.example
README.md

## sample_output.txt
SAMPLE OUTPUT / DEMONSTRATION
```

Input:

Name: Anu

User ID: FB001

Age: 18

Weight: 60 kg

Goal: Weight Loss

Intensity: Medium

Output:

7-DAY PERSONALIZED WORKOUT PLAN

Day 1 – Full Body

Warm-up: 5–10 minutes brisk walking and mobility

Main workout: Squats 3x12, Push-ups 3x10, Glute bridges 3x15

Cooldown: 5 minutes stretching

...

Day 7 – Active Recovery

Easy walk, gentle stretching and recovery.

Nutrition Tip:

Prioritize balanced meals, adequate protein, vegetables, whole grains, and regular hydration.

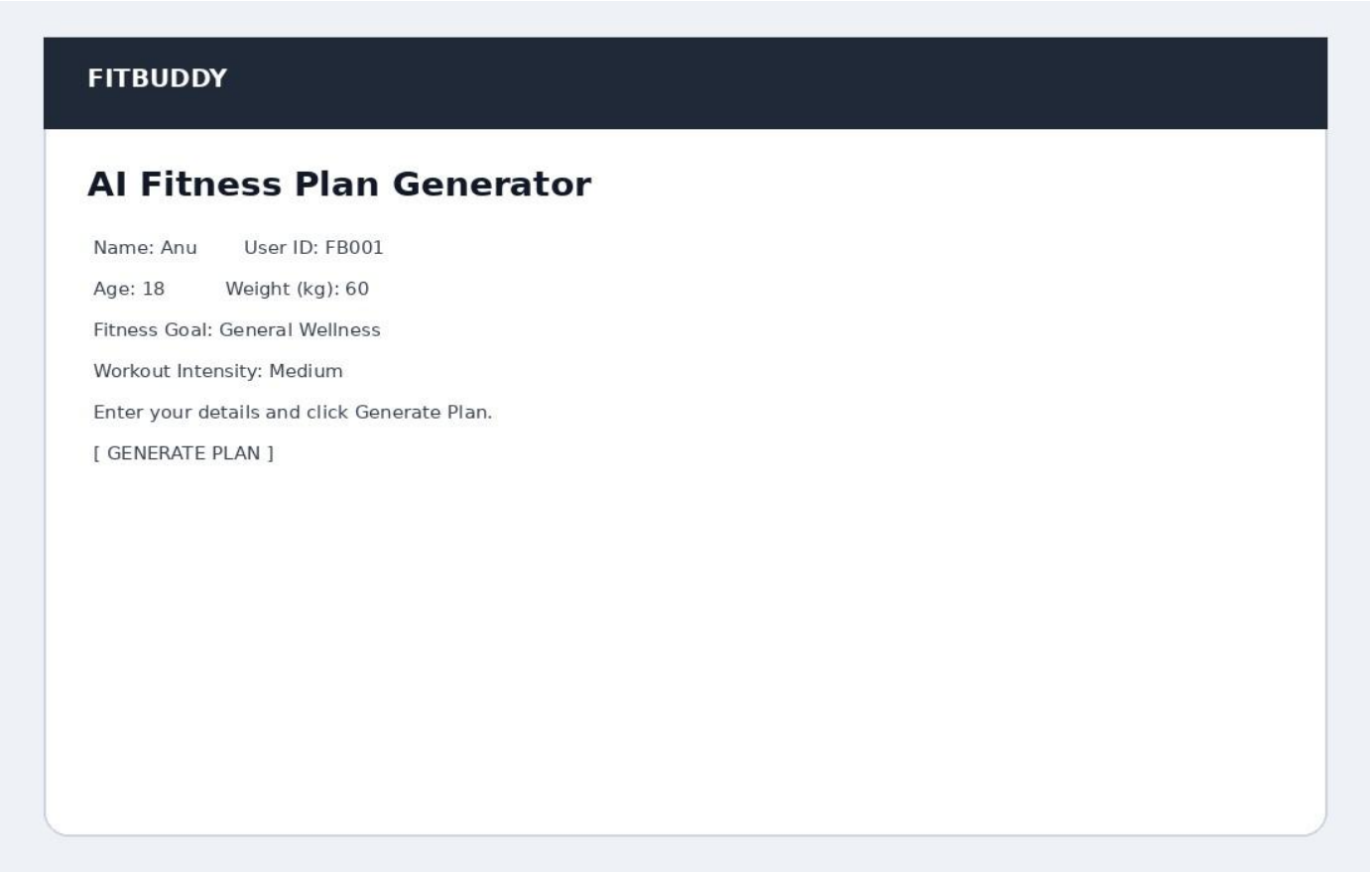
Feedback:

"Include more cardio and one additional rest period."

Updated Output:

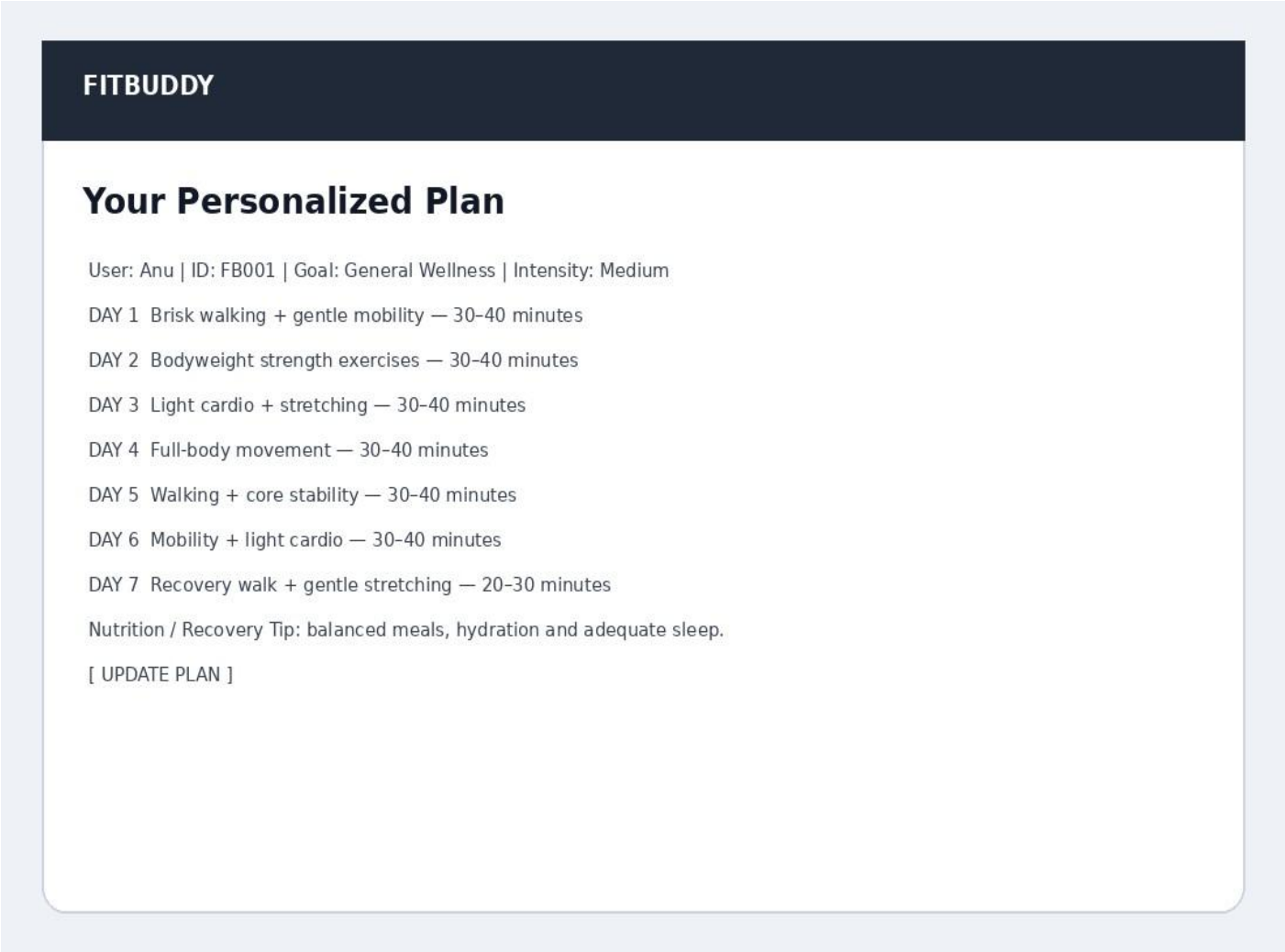
The plan is revised to incorporate the requested cardio focus and recovery preference.

22. Demonstration Output Screens Output 1. Home / input page



Result / generated-plan page

Output 2 – Generated Personalized Plan



Output 3 – Admin / All Users Page

FITBUDDY

View All Users

User ID: FB001 | Name: Anu | Age: 18 | Weight: 60 kg

Goal: General Wellness | Intensity: Medium

Original Plan: Available

Updated Plan: Available after feedback

Admin access uses the configured ADMIN_KEY.